

Key takeaways

Phạm vi của biến

Phạm vi (scope) là khu vực hoạt động của biến.

Trong Javascript, phạm vi được thể hiện thông qua các cặp ngoặc nhọn (`{}`).

Đoạn code không nằm trong cặp ngoặc nhọn nào gọi là phạm vi global (toàn cục).

Điều kiện if...else...

Cú pháp điều kiện if...else...:

```
if (condition) {  
    // code block 1  
} else {  
    // code block 2  
}
```

Trong đó:

- Nếu `condition = true`, sẽ chạy logic ở code block 1.
- Nếu `condition = false`, sẽ chạy logic ở code block 2.

Có thể hiểu đơn giản, điều kiện if...else là điều kiện dạng nếu...thì... Nếu đúng thì chạy logic ở block của nếu. Nếu không đúng thì chạy logic ở block của thì...

Điều kiện if...else if...

Cú pháp điều kiện if...else if...:

```
if (condition) {  
    // Code block 1  
} else if (condition2) {  
    // Code block 2  
} else if (condition3) {  
    // Code block 3  
} else {  
    // Code block 4  
}
```

Có thể hiểu if...else if là câu điều kiện nâng cao hơn câu điều kiện if, thêm một lần kiểm tra điều kiện, chia thành nhiều trường hợp:

- Nếu `condition` đúng, chạy logic trong code block 1
- Nếu `condition` sai, kiểm tra `condition2`
 - Nếu `condition2` đúng, chạy logic trong code block 2
 - Nếu `condition2` sai, kiểm tra `condition3`
 - Nếu `condition3` đúng, chạy logic trong code block 3
 - Nếu `condition3` sai, chạy logic trong code block 4

Có thể hiểu, câu lệnh else là “các trường hợp còn lại”.

Điều kiện switch...case

`switch...case` thường dùng để rẽ nhánh trong trường hợp có nhiều điều kiện khác nhau. `switch...case` giúp code trở nên gọn gàng, dễ nhìn hơn.

Cú pháp switch...case:

```
switch (condition) {  
  case "<case_value_1>":  
    // code block 1  
    break;  
  case "<case_value_2>":  
    // code block 2  
    break;  
  case "<case_value_3>":  
    // code block 3  
    break;  
  default:  
    // code block 4  
}
```

Trong đó:

- `condition` gọi là biểu thức điều kiện
- Các `case_value1`, `case_value2`, `case_value3`... được gọi là các giá trị của các trường hợp.
- Nếu sau khi tính toán, giá trị của `condition` rơi vào `case_value` nào thì sẽ chạy đoạn code tương ứng với case đó.

- Lưu ý, mỗi case cần có **break** để thoát khỏi đoạn logic của case. Nếu không có break, đoạn logic của case phía dưới sẽ tiếp tục được chạy.
- **default** là case mặc định. Trong trường hợp nếu không có case nào match, logic default sẽ được chạy.

Xét ví dụ: kiểm tra biến **day**, nếu giá trị 2 thì in ra "thứ 2", giá trị 3 thì in ra thứ 3, ..., giá trị 7 in ra "thứ 7".

Nếu với lệnh if, ta sẽ làm như sau:

```
const day = 3;
```

```
if (day === 2) {  
    console.log("Thứ 2");  
} else if (day === 3) {  
    console.log("Thứ 3");  
} else if (day === 4) {  
    console.log("Thứ 4");  
} else if (day === 5) {  
    console.log("Thứ 5");  
} else if (day === 6) {  
    console.log("Thứ 6");  
} else if (day === 7) {  
    console.log("Thứ 7");  
} else {  
    console.log("Chủ nhật");  
}
```

Có thể thấy, với if..else if... trông khá rối.
Viết lại đoạn code trên sử dụng switch...case:

```
const day = 3;
```

```
switch (day) {  
    case 2:  
        console.log("Thứ 2");  
        break;  
    case 3:  
        console.log("Thứ 3");  
}
```

```
        break;
    case 4:
        console.log("Thứ 4");
        break;
    case 5:
        console.log("Thứ 5");
        break;
    case 6:
        console.log("Thứ 6");
        break;
    case 7:
        console.log("Thứ 7");
        break;
    default:
        console.log("Chủ nhật");
}
```

Logic lúc này trông gọn gàng và sáng hơn nhiều.

Hãy sử dụng `switch...case...` trong trường hợp có nhiều trường hợp bạn nhé. ^^

Toán tử so sánh

Toán tử so sánh dùng để so sánh giá trị của 2 vế. Toán tử so sánh sẽ trả về true nếu đúng, trả về false nếu sai.

Toán tử	Ví dụ	Kết quả	Ý nghĩa
>	2 > 5	false	Phép so sánh lớn hơn
<	2 < 5	true	Phép so sánh nhỏ hơn
>=	2 >= 5	false	Phép so sánh lớn hơn hoặc bằng
<=	2 <= 5	true	Phép so sánh nhỏ hơn hoặc bằng
==	5 == '5' 2 == 2	true true	Phép so sánh bằng, chỉ so sánh giá trị, không so sánh về kiểu dữ liệu

===	5 === '5' '5' === '5'	false true	Phép so sánh bằng, so sánh cả về giá trị và về kiểu dữ liệu
!=	5 != '5' 2 != 3	false true	Phép so sánh khác, chỉ so sánh về giá trị, không so sánh về kiểu dữ liệu
!==	5 !== '5' '5' !== '5'	true false	Phép so sánh khác, so sánh cả về giá trị và về kiểu dữ liệu

Vòng lặp forEach

Cú pháp

`<biến_là_tên_mảng>.forEach(callbackFn)`

Trong đó:

- `<biến_là_tên_mảng>`: tên của biến
- `callbackFn`: viết tắt của callback function, là hàm xử lý vòng for. Thường hàm này sẽ là: `item => { // code ở đây }`.

Ví dụ

```
let numberArr = [1, 20.5, -300, 4];
numberArr.forEach(number => {
  console.log(number)
})
```

Giải thích:

- `numberArr`: tên biến
- `number => { console.log(number) }`: hàm callback chứa thông tin mảng.
 - Chú ý tới tên biến `number`, biến này đại diện cho các phần tử trong mảng. Ở mỗi vòng lặp, biến `number` sẽ được gán giá trị của từng phần tử trong mảng.

Tips sử dụng

- Vòng lặp `forEach` dùng để lặp các phần tử trong mảng, khi bạn không cần lấy thứ tự của các phần tử.
- Trong callback function, nếu chỉ có 1 dòng duy nhất, có thể bỏ cặp ngoặc nhọn bao ngoài:

Đầy đủ:

```
numberArr.forEach(number => {
```

```
        console.log(number)
    })
```

Rút gọn:

```
numberArr.forEach(number => console.log(number));
```

-

Vòng lặp for...in

Cú pháp

```
for (const property in object) {
    // Code ở đây
}
```

Trong đó:

- **const property**: khai báo một biến tên là property. Trước mỗi vòng lặp, biến property này sẽ có giá trị là các thuộc tính của object
- **object**: là tên biến có kiểu dữ liệu object

Ví dụ

```
const student = {
    id: 1,
    name: "Alex",
    isGraduated: true
};

for (const property in student) {
    console.log(property);
}

// Kết quả:
// id
// name
// isGraduated
```

Giải thích:

- object student có 3 thuộc tính: `id`, `name`, `isGraduated`.
- Vòng lặp `for...in` chạy, mỗi vòng lặp sẽ gán lần lượt giá trị của các thuộc tính `id`, `name`, `isGraduated` vào hằng số property

Tips sử dụng

- Vòng lặp `for..in` thường được sử dụng khi bạn muốn lặp trong các thuộc tính của object.
- Có thể kết hợp với cú pháp truy xuất thuộc tính của object để lấy ra giá trị tương ứng

```
for (const property in student) {  
    console.log(`Property: ${property}, value: ${student[property]}`);  
}
```
- Thứ tự lấy ra các thuộc tính phụ thuộc vào loại trình duyệt mà bạn đang chạy. Vì vậy, bạn chỉ nên dùng `for...in` trong trường hợp không quan tâm tới thứ tự chạy.

Vòng lặp `for...of`

Cú pháp

```
for (const item of <tên_biến_của_mảng>) {  
    // Code ở đây  
}
```

Trong đó:

- `item` là tên của hằng số, giá trị hằng số sẽ được khởi tạo trong mỗi lần lặp
- `<tên_biến_của_mảng>` là tên biến chứa mảng.

Ví dụ

```
const arr = [1, 3, 5, 2];  
for (const item of arr) {  
    console.log (item);  
}
```

```
// Kết quả:  
// 1  
// 3  
// 5  
// 2
```

Giải thích:

- arr là mảng chứa các phần tử 1, 3, 5, 2
- Trong vòng lặp for...of, ta khởi tạo hằng số item.
 - Mỗi lần vòng lặp chạy, item sẽ lấy giá trị lần lượt của từng phần tử: 1, 3, 5, 2.
 - Trong vòng lặp, ta sử dụng giá trị của các biến này.

Tips sử dụng

- Lặp qua các giá trị của mảng: for...of là cách đơn giản và rõ ràng để lặp qua tất cả các giá trị trong một mảng, đặc biệt khi bạn không cần quan tâm đến chỉ số của từng phần tử.
- Lặp qua các chuỗi: Chuỗi trong JavaScript cũng có thể được coi là một dãy các ký tự, vì vậy bạn có thể sử dụng for...of để lặp qua từng ký tự trong chuỗi.

Ví dụ:

```
const str = "Hello";  
for (const char of str) {  
    console.log(char);  
}
```

// Kết quả:

// H

// e

// l

// l

// o

Break

Cú pháp

Break dùng để thoát khỏi vòng lặp hoặc mệnh đề switch. Thường kết hợp với một điều kiện để dừng vòng lặp sớm hơn khi đã thỏa mãn điều kiện, hoặc

```
for(let i = 0; i < 100; i++) {  
    // some code ...  
    break;  
    //....  
}
```


Giải thích:

trong ví dụ trên, break sẽ thoát khỏi vòng lặp ngay khi gặp break;

Ví dụ

Xét bài toán sau: lặp các số từ 1 đến 100. Mỗi lần lặp, cộng dồn giá trị vào biến sum sao cho $sum \leq 20$

Không có break

```
let sum = 0;
for(let i = 0; i < 100; i++) {
    if (sum <= 20) {
        sum += i;
    }
}
console.log("Sum without break:", sum);
```

Giải thích:

- Khi không dùng break, ta bắt buộc phải chạy vòng lặp hết 100 lần.
- Ở mỗi vòng lặp, kiểm tra nếu tổng hiện tại nhỏ hơn hoặc bằng 20 thì cộng i vào sum. Tuy nhiên, vòng lặp vẫn sẽ chạy hết 100 lần, dù sum đã vượt quá 20 từ lâu.

Có break

```
let sum = 0;
for (let i = 0; i < 100; i++) {
    if (sum + i > 20) {
        break; // Thoát khỏi vòng lặp nếu cộng i vào sum sẽ vượt
        quá 20
    }
}
```

```
    sum += i;
}
console.log("Sum with break:", sum);
```

Giải thích:

- Với break, vòng lặp sẽ dừng lại ngay khi tổng sum + i lớn hơn 20.
- Điều này giúp tiết kiệm tài nguyên và thời gian thực thi, đặc biệt khi vòng lặp có số lượng lớn các lần lặp.
- Trong trường hợp này, vòng lặp sẽ không chạy hết 100 lần.

Tips

- **break** thường được dùng khi bạn muốn tìm kiếm một giá trị cụ thể trong một mảng hoặc một tập hợp dữ liệu, và bạn muốn dừng lại ngay khi tìm thấy giá trị đó.
- Sử dụng **break** giúp tránh việc thực hiện các phép tính không cần thiết sau khi đã đạt được mục tiêu.
- Khi sử dụng **break** trong các vòng lặp lồng nhau, nó chỉ thoát khỏi vòng lặp gần nhất chứa nó.

Continue

Cú pháp

```
continue;
```

Ví dụ

Có continue

```
for (let i = 0; i < 10; i++) {
  if (i % 2 === 0) { // Nếu i là số chẵn
    continue; // Bỏ qua các lệnh còn lại trong vòng lặp hiện tại và chuyển sang lần lặp tiếp theo
  }
  console.log(i); // Chỉ in ra các số lẻ
}
```

// Kết quả:

```
// 1
// 3
// 5
// 7
// 9
```

Không có continue

```
for (let i = 0; i < 10; i++) {
  if (i % 2 === 0) {
    // Không có continue ở đây
  } else {
    console.log(i);
  }
}
// Kết quả:
// 1
// 3
// 5
// 7
// 9
```

Giải thích:

- `continue` dùng để bỏ qua phần còn lại của vòng lặp hiện tại và chuyển sang lần lặp tiếp theo.
- Trong ví dụ trên, khi `i` là số chẵn, `continue` được gọi, khiến cho `console.log(i)` bị bỏ qua, do đó chỉ các số lẻ được in ra.

Tips

- `continue` giúp bạn bỏ qua một số trường hợp cụ thể trong vòng lặp mà không cần thoát khỏi vòng lặp hoàn toàn.

String util functions

1. `trim()`

- **Khái niệm:** Phương thức `trim()` dùng để loại bỏ khoảng trắng ở đầu và cuối chuỗi. Khoảng trắng bao gồm cả dấu cách, dấu tab, và các ký tự không in khác.

Ví dụ:

```
let str = "   JavaScript is awesome! ";  
str = str.trim();  
console.log(str); // Kết quả: "JavaScript is awesome!"
```

2. `toLowerCase()` và `toUpperCase()`

- **Khái niệm:**
 - `toLowerCase()` chuyển đổi tất cả các ký tự trong chuỗi thành chữ thường.
 - `toUpperCase()` chuyển đổi tất cả các ký tự trong chuỗi thành chữ hoa.

Ví dụ:

```
let str = "JavaScript is awesome!";  
console.log(str.toLowerCase());  
// Kết quả: "javascript is awesome!"  
console.log(str.toUpperCase());
```

3. `includes()`

- **Khái niệm:**
 - Phương thức `includes()` kiểm tra xem một chuỗi có chứa một chuỗi con (substring) hay không.
 - Nó trả về `true` nếu tìm thấy và `false` nếu không.

Ví dụ:

```
let str = "JavaScript is awesome!";  
console.log(str.includes("awesome")); // Kết quả: true  
console.log(str.includes("Awesome")); // Kết quả: false
```

4. `replace()`

- **Khái niệm:**
 - Phương thức `replace()` dùng để thay thế một chuỗi con trong chuỗi bằng một chuỗi khác.
 - Bạn có thể thay thế chỉ chuỗi đầu tiên hoặc tất cả chuỗi con bằng cách sử dụng biểu thức chính quy.

Ví dụ:

```
let str = "JavaScript is awesome!";  
str = str.replace("awesome", "fun");  
console.log(str); // Kết quả: "javascript is fun!"
```

5. `split()`

- **Khái niệm:** Phương thức `split()` chia một chuỗi thành một mảng các chuỗi con, dựa trên một ký tự (delimiter).

Ví dụ:

```
let str = "JavaScript is awesome!";  
let words = str.split(" ");  
console.log(words); // Kết quả: ["JavaScript", "is", "awesome!"]
```

6. `substring()`

- **Khái niệm:** Phương thức `substring()` trả về một phần của chuỗi, bắt đầu từ chỉ số (index) được chỉ định đến một chỉ số khác hoặc đến cuối chuỗi.

Ví dụ:

```
let str = "Hello World!";
```

```
console.log(str.substring(0, 5)); // Kết quả: "Hello"  
console.log(str.substring(6));    // Kết quả: "World!"
```

7. indexOf()

- **Khái niệm:** Phương thức `indexOf()` trả về vị trí xuất hiện đầu tiên của một chuỗi con trong chuỗi, hoặc `-1` nếu không tìm thấy.

Ví dụ:

```
let str = "Hello World!";  
  
console.log(str.indexOf("World")); // Kết quả: 6  
  
console.log(str.indexOf("JavaScript")); // Kết quả: -1
```

Array util functions

1. map()

- **Khái niệm:** Phương thức `map()` tạo ra **một mảng mới** bằng cách áp dụng một hàm lên từng phần tử của mảng gốc.

Ví dụ:

```
let numbers = [1, 2, 3, 4];  
let squared = numbers.map(num => num * 2);  
console.log(squared); // Kết quả: [2, 4, 6, 8]
```

2. filter()

- **Khái niệm:** Phương thức `filter()` tạo ra **một mảng mới** chỉ bao gồm các phần tử thỏa mãn điều kiện được chỉ định trong hàm callback.

Ví dụ:

```
let numbers = [1, 2, 3, 4];
```

```
let evenNumbers = numbers.filter(num => num % 2 === 0);  
console.log(evenNumbers); // Kết quả: [2, 4]
```

3. find()

- **Khái niệm:** Phương thức `find()` trả về giá trị của phần tử đầu tiên trong mảng thỏa mãn điều kiện được chỉ định trong hàm callback, nếu không có phần tử nào thỏa mãn thì trả về `undefined`.

Ví dụ:

```
let numbers = [1, 2, 3, 4];  
let firstEven = numbers.find(num => num % 2 === 0);  
console.log(firstEven); // Kết quả: 2
```

4. reduce()

- **Khái niệm:** Phương thức `reduce()` áp dụng một hàm lên từng phần tử của mảng (từ trái qua phải) để trả về một giá trị duy nhất.

Ví dụ:

```
let numbers = [1, 2, 3, 4];  
let sum = numbers.reduce((total, num) => total + num, 0);  
console.log(sum); // Kết quả: 10
```

```
//total: biến nhận giá trị duy nhất  
//num: phần tử của mảng  
//0 giá trị khởi tạo cho biến total
```

5. some()

- **Khái niệm:** Phương thức `some()` kiểm tra xem có ít nhất một phần tử trong mảng thỏa mãn điều kiện được chỉ định trong hàm callback. Trả về `true` nếu tìm thấy, ngược lại trả về `false`.

Ví dụ:

```
let numbers = [1, 2, 3, 4];  
let hasEven = numbers.some(num => num % 2 === 0);  
console.log(hasEven); // Kết quả: true
```

6. every()

- **Khái niệm:** Phương thức `every()` kiểm tra xem **tất cả** các phần tử trong mảng có thỏa mãn điều kiện được chỉ định trong hàm callback hay không. Trả về `true` nếu **tất cả** đều thỏa mãn, ngược lại trả về `false`.

Ví dụ:

```
let numbers = [2, 4, 6, 8];  
let allEven = numbers.every(num => num % 2 === 0);  
console.log(allEven); // Kết quả: true
```

7. push()

- **Khái niệm:** Phương thức `push()` **thêm một hoặc nhiều phần tử vào cuối mảng** và trả về độ dài mới của mảng.

Ví dụ:

```
let numbers = [1, 2, 3];  
numbers.push(4, 5);  
console.log(numbers); // Kết quả: [1, 2, 3, 4, 5]
```

8. shift()

- **Khái niệm:** Phương thức `shift()` **loại bỏ phần tử đầu tiên của mảng và trả về phần tử bị loại bỏ**. Phương thức này thay đổi độ dài của mảng**.

Ví dụ:

```
let numbers = [1, 2, 3];  
let firstElement = numbers.shift();
```



```
console.log(firstElement); // Kết quả: 1  
console.log(numbers); // Kết quả: [2, 3]
```

9. `sort()`

- **Khái niệm:** Phương thức `sort()` **sắp xếp các phần tử của mảng** theo thứ tự tăng dần hoặc theo hàm so sánh được cung cấp. Nó thay đổi mảng ban đầu.
- `sort()` mặc định: **sắp xếp các phần tử của mảng như chuỗi theo thứ tự từ điển học Unicode (ASCII hoặc UTF-16)**. Điều này có nghĩa là mỗi phần tử sẽ được **chuyển đổi thành chuỗi** (nếu không phải là chuỗi) trước khi so sánh.

Ví dụ:

```
let numbers = [4, 2, 3, 1];  
  
numbers.sort();  
  
console.log(numbers); // Kết quả: [1, 2, 3, 4]
```